

Deep Learning and Transformers: A Practical Map

Start with a small neural model and a transparent training loop before studying transformers. Connect tensors, losses, gradients, optimization, and validation. Then explain attention, tokens, context, and generation as computational mechanisms with costs and limitations, not as magical understanding.

For: Engineers who want to explain neural-network training, embeddings, attention, and inference constraints without confusing model terminology with verified understanding.

Suggested pace: 4-6 weeks after ML foundations | Python, linear algebra, and basic gradients

What you will be able to do

- Trace a forward pass, loss, gradient, and parameter update.
- Recognize optimization problems, overfitting, and data leakage separately.
- Explain embeddings, attention, masking, and autoregressive generation.
- Discuss context, batching, memory, and evaluation trade-offs during inference.

Build the right mental model

Training changes parameters through a loss

A forward pass produces predictions, a loss measures disagreement with the objective, and backpropagation computes derivatives used by an optimizer. Lower training loss alone does not establish useful generalization. Keep validation and data contracts visible throughout the process.

Tensor meaning precedes shape manipulation

Track batch, sequence, feature, and head dimensions explicitly. Broadcasting or reshaping can produce valid-looking tensors with the wrong semantic alignment. Test a tiny case where the intended result can be inspected directly before scaling up.

Attention is a weighted information operation

Queries and keys produce compatibility scores, normalized weights combine value representations, and masks restrict allowed context. Ordinary full attention includes a quadratic pairwise score structure in sequence length, though optimized implementations change memory behavior. A larger context does not guarantee correct use of every fact.

Study with evidence.

Use an established framework and small datasets. Inspect losses, shapes, gradients, and sample predictions. Learn to overfit a tiny clean dataset as a debugging check, then evaluate generalization on properly isolated data instead of assuming a large model will hide pipeline errors.

Week 1: Small neural models

Implement a simple regression or classification model with a clear loss and optimizer. Verify tensor dimensions and compare one gradient with a numerical approximation on a tiny example. Inspect learning curves.

Deliverable and check: A training loop whose inputs, target, loss, and update can be explained line by line.

Week 2: Generalization and optimization

Study activations, initialization, normalization, regularization, learning rate, and early stopping. Distinguish unstable optimization from overfitting. Keep the test set outside repeated model selection.

Deliverable and check: An experiment log that changes one factor at a time and reports validation behavior.

Weeks 3-4: Transformer mechanisms

Study tokenization, embeddings, positional information, attention, residual paths, masking, and next-token prediction. Trace a small sequence and explain why causal masking differs from unrestricted encoder attention.

Deliverable and check: A diagram with meaningful tensor axes and an explanation of which positions may influence a prediction.

Weeks 5-6: Inference and adaptation

Compare prompting, retrieval, and fine-tuning for different needs. Study batching, context/output budgets, KV cache, quantization, and latency. Evaluate a small application with representative failure cases rather than only a pleasing demo.

Deliverable and check: A measured inference experiment and an honest statement of quality, cost, and remaining limits.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Check a gradient before trusting a training pipeline

Consider the scalar loss $L(w) = (w - 3)^2$. Starting at $w=0$, use a learning rate of 0.1 to understand one gradient-descent step before relying on automatic differentiation in a larger model.

A worked approach

1. The derivative is $2*(w-3)$, so at $w=0$ it is -6. Gradient descent subtracts learning_rate times gradient, producing $w=0.6$. The loss drops from 9 to $(0.6-3)^2 = 5.76$.
2. Approximate the derivative with a small symmetric finite difference and compare its sign and magnitude with the analytic value. Choose an appropriate step size and numerical tolerance; extremely tiny differences can lose precision.
3. Repeat the idea on a tiny framework model and inspect one parameter's gradient. Confirm the target shape, loss reduction, and update direction before interpreting a long training run or adding more layers.

Expected result

The exercise makes optimization concrete and provides a debugging habit: validate a small known case before scaling. It does not claim that every neural-network loss is convex or that one learning rate works universally.

Verify the result

- Does the update move in the direction that reduces this loss?
- Are gradients and parameter shapes aligned?
- Can a tiny model fit a clean tiny dataset before larger experiments begin?

Extend the exercise

Increase the learning rate in the scalar example and observe overshoot or divergence. Then explain why a low training loss and a low validation loss answer different questions about a real model.

Make the learning stick.

Does attention prove that a model understands a document?

No. Attention is a computational mechanism, not a guarantee of factual reasoning or faithful use of context. Evaluate the resulting behavior and claims against evidence, including cases where relevant information is present but ignored or misused.

Why is inference memory more than model weights?

Activations, KV cache, batching, allocator overhead, and framework behavior also consume memory. Longer sequences and more simultaneous requests can dominate capacity even when the weights fit on the device.

When should I fine-tune?

When a stable task or behavior and suitable training/evaluation data justify it. Frequently changing facts and per-user document access often need retrieval or external data. Fine-tuning is not a universal fix for missing knowledge or poor evaluation.

Avoid these traps

- Manipulating tensor shapes without tracking their meaning.
- Equating lower training loss with better real-world behavior.
- Claiming context length or model size guarantees factual accuracy.

Ready for the next step?

- A small training step and its gradient can be explained.
- Data splits and model-selection boundaries are valid.
- Attention and masking are described operationally.
- Inference costs and failure cases are measured and stated.

Reliable next reads

- [PyTorch tutorials](#)
- [Google ML neural networks](#)
- [Hugging Face LLM course](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.